Explore, Then Exploit: A Two-Phase Framework for Credit-Aware Tree Rollouts in LLM Reasoning

Mani Tej Sriram **Adithya Casichetty Addepalli** **Likhit Bhogadi** **Sanyam Agrawal**
International Institute of Information Technology Hyderabad

Abstract

Reinforcement learning (RL) for large language model (LLM) reasoning usually rewards only the final answer, making it hard to tell which intermediate decisions caused the outcome. Tree-based rollouts help by sharing reasoning prefixes across trajectories, but under a fixed rollout budget the question of *where* to branch remains open. We propose a two-phase framework for exploration–exploitation tree search in LLM reasoning: a budget of n rollouts is split into an exploration phase ($n - k$ rollouts) and an exploitation phase (k rollouts), each defined by its own goal and its own scoring metric, so either phase can be replaced independently. In our instantiation, Phase I builds an exploratory tree with TreeRL's surprisal-guided expansion, and Phase II spends the remaining rollouts on states that score highest on a weighted combination of novelty, the probability that a new rollout yields an unseen continuation, and consequentiality, the change in estimated value across a decision. With Qwen2.5-Math-1.5B-Instruct and the same 30-rollout budget as TreeRL, our method reaches **48.55%** greedy accuracy on 1,755 math problems, ahead of TreeRL (48.21%) and ChainRL (47.52%), and all four of our configurations beat both baselines at step 100. Our code is open-sourced at https://github.com/likhitbhogadi/TreeRL.

1 Introduction

Reinforcement learning (RL) has become an effective approach for improving the reasoning capabilities of large language models (LLMs). However, reasoning tasks often provide feedback only through a terminal reward, such as whether the final answer is correct. This creates a fundamental *credit assignment* problem: a long reasoning trajectory may contain many intermediate decisions, while the final reward does not indicate which decisions were responsible for the outcome.

A natural way to obtain more localized credit is to generate multiple trajectories that share intermediate reasoning states. When trajectories diverge from the same state and receive different terminal rewards, their outcomes provide counterfactual evidence about the consequences of decisions at that state. Recent methods exploit this structure for credit assignment and exploration, including TEMPO (Tran et al., 2025), TreePO (Li et al., 2025), and TreeRL (Hou et al., 2025). Other works investigate adaptive rollout allocation (Zhang et al., 2026; Zou et al., 2026) and the use of counterfactual reasoning paths for reducing credit-assignment variance (Ding et al., 2026).

However, generating trajectories can be computationally expensive, making the rollout budget a limited resource. This motivates the following question:

> *Given a fixed rollout budget, where should additional trajectories be generated to obtain the most useful counterfactual evidence for credit assignment?*

Generating all trajectories according to a single exploration criterion may spend computation on states that are uncertain but do not benefit substantially from additional sampling. Conversely, immediately allocating rollouts according to observed rewards may overexploit states before sufficient alternative outcomes have been explored.

We therefore propose a two-stage strategy that explicitly separates *exploration* from *exploitation*. The first stage uses $n - k$ trajectories to construct an exploratory reasoning tree using surprisal, following the uncertainty-driven expansion principle of TreeRL (Hou et al., 2025). The second stage uses the resulting tree to allocate the remaining k trajectories. Each state is scored by a weighted combination of its novelty, the probability that additional sampling produces a continuation not yet observed, and its consequentiality, the observed

change in value across the decision leading to it. Rather than filtering candidates by novelty alone, this joint score lets a state with moderate novelty but high consequentiality still receive additional rollouts.

2 Preliminaries

2.1 Tree-Based Reasoning and Credit Assignment

Tree-based rollout methods represent multiple reasoning trajectories as a shared tree. Each internal node corresponds to an intermediate reasoning state, while different children represent alternative continuations from that state. This structure makes it possible to compare trajectories that share a common reasoning prefix.

Consider a reasoning trajectory $\tau = (v_1, \ldots, v_m)$, where each v_i represents a reasoning segment and the trajectory receives a terminal reward $R(\tau)$. For a node v, let $\mathcal{L}(v)$ denote its descendant leaf trajectories. A simple empirical value estimate is

$$V(v) = \frac{1}{|\mathcal{L}(v)|} \sum_{\ell \in \mathcal{L}(v)} R(l). \tag{1}$$

When multiple trajectories share v, this value summarizes the observed outcomes associated with that reasoning state. The same tree structure can therefore be used both to obtain localized credit signals and to determine where additional rollout computation should be spent.

3 Related Work

Tree-based reasoning and credit assignment. TEMPO (Tran et al., 2025) exploits shared tree structure to estimate intermediate values from descendant outcomes and obtain more localized credit signals. TreePO (Li et al., 2025) uses segment-level tree rollouts for reasoning optimization, while TreeRL (Hou et al., 2025) performs on-policy tree search by expanding uncertain intermediate states. Our method similarly uses a shared reasoning tree, but explicitly divides the rollout budget between an exploratory stage and a subsequent targeted expansion stage.

Adaptive rollout allocation. IGRPO (Zhang et al., 2026) investigates information-gain-based adaptive rollout allocation in a tree-structured rollout framework, while TRACE (Zou et al., 2026) studies adaptive allocation of a fixed rollout budget. These works motivate the idea that rollout computation should be allocated selectively rather than uniformly. Our approach similarly treats rollouts as a limited resource, but uses a two-stage procedure in which the first stage builds an exploratory tree and the second stage uses novelty and observed consequentiality to allocate additional trajectories.

Counterfactual reasoning paths. The work *Reducing Credit-Assignment Variance Through Counterfactual Reasoning Paths* (Ding et al., 2026) investigates how alternative reasoning paths can provide useful information for reducing variance in credit assignment. Our approach addresses the complementary question of where such additional counterfactual paths should be generated when only a fixed number of rollouts is available.

4 Proposed Method

4.1 Problem Formulation

For each problem x, let n denote the total rollout budget. We divide it into two portions $(n-k, k)$. The first $n-k$ trajectories are used for *exploration*, while the remaining k trajectories are reserved for *credit-guided counterfactual expansion*.

4.2 Phase I: Entropy-Guided Exploration

The first phase constructs an initial tree T_0 using $n-k$ trajectories. The goal of this phase is exploration rather than exploitation. Terminal rewards are therefore not used to determine where trajectories are generated.

Instead, we follow the uncertainty-driven expansion principle of TreeRL (Hou et al., 2025), using local token-level surprisal to identify reasoning states with high uncertainty.

Methodology: We construct the initial reasoning tree T_0 following the standard TreeRL procedure. Starting from the original problem x, reasoning is generated autoregressively, with each reasoning segment represented as a node in the tree. Rather than generating independent complete trajectories from the root, TreeRL identifies promising intermediate reasoning states at which additional rollouts can be initiated, allowing multiple continuations to share their common prefix.

To identify branching locations, TreeRL uses the surprisal of the next generated token. Let c_v denote the model context at reasoning state v, and let a be the selected next token. The surprisal is defined as $H(v) = -\log p_\theta(a \mid c_v)$, where $p_\theta(a \mid c_v)$ is the

probability assigned to the selected token by the model. A high surprisal indicates that the model assigns relatively low probability to the selected continuation, making the corresponding reasoning state a potential location for further exploration.

At each rollout, an eligible reasoning state is selected based on this criterion, and a complete continuation is generated from that state. The new trajectory shares the existing prefix up to the selected state and introduces a new branch thereafter. This process is repeated until $n - k$ trajectories have been generated, resulting in the initial tree T_0. Importantly, no terminal reward is used to select branching locations during this phase; the initial tree is constructed solely from the model's generation behavior.

4.3 Phase II: Novelty-Aware Credit-Guided Expansion

After the exploratory tree has been constructed, terminal rewards become available. The remaining k rollouts are reserved for targeted counterfactual expansion.

The goal of Phase II is to identify reasoning states where another rollout is both likely to produce a novel continuation and potentially consequential. We therefore combine novelty and consequentiality into a joint score used to guide rollout allocation.

4.3.1 Probability-Based Novelty

As in EPTree, a node v is a token position in the reasoning tree. Let c_v denote its model context, $\mathcal{L}(v)$ its observed descendant leaf trajectories, and $\mathcal{E}_v$ the set of distinct continuations observed from v, each ending in a termination token such as EOS or at the maximum response length if truncated.

The probability of a complete continuation vanishes as its length grows, so the probability of differing from all observed continuations approaches 1 almost everywhere and carries little information. We therefore measure novelty over a horizon of h tokens. For $Y_i = (y_{i,1}, \ldots, y_{i,T_i}) \in \mathcal{E}_v$, where $y_{i,t}$ is the t-th token of the i-th observed continuation, T_i its length, and $y_{i,<t} = (y_{i,1}, \ldots, y_{i,t-1})$ the tokens preceding it, let $Y_i^{(h)}$ denote its first $\min(h, T_i)$ tokens, with probability

$$P\big(Y_i^{(h)} \mid c_v\big) = \prod_{t=1}^{\min(h,T_i)} p_\theta(y_{i,t} \mid c_v, y_{i,<t}). \quad (2)$$

Letting $\mathcal{E}_v^h$ be the set of distinct such prefixes, the novelty score of node v is

$$\boxed{N_h(v) = 1 - \sum_{Y \in \mathcal{E}_v^h} P(Y \mid c_v).} \quad (3)$$

$N_h(v)$ estimates the probability that a new continuation from v differs from every observed continuation within its next h tokens. Short prefixes cover a measurable share of the probability mass, so each rollout from v reduces the score or leaves it unchanged, and the complete-continuation form is recovered when h is at least the length of every continuation in $\mathcal{E}_v$. The score measures exact sequence novelty rather than semantic novelty: continuations with different tokens are distinct even if they express equivalent reasoning.

4.3.2 Local Consequentiality

Let $\mathcal{V}(T_0)$ denote the set of eligible candidate nodes, defined as all non-leaf nodes of T_0. For each candidate node $v \in \mathcal{V}(T_0)$, we measure the change in estimated value across the reasoning decision leading to that node:

$$\delta(v) = V(v) - V(\mathrm{parent}(v)). \quad (4)$$

The parent state represents the reasoning state at which the decision leading to v was made. Thus, $\delta(v)$ provides a local signal for how the observed outcome value changes across that decision.

We use the magnitude of this difference as the consequentiality score:

$$C(v) = |\delta(v)|. \quad (5)$$

A larger value indicates that the observed outcomes differ more substantially across the corresponding decision boundary. We take the magnitude because a decision that sharply lowers the value is as informative for credit assignment as one that sharply raises it.

4.3.3 Joint Novelty–Consequentiality Scoring

Rather than filtering candidate nodes using novelty alone, we combine novelty and consequentiality into a single score. This allows a node with moderate novelty but high consequentiality to remain a candidate for additional exploration.

Let $\mathcal{P}(T_0)$ denote the set of distinct parent states associated with eligible candidate nodes:

$$\mathcal{P}(T_0) = \{\mathrm{parent}(v) \mid v \in \mathcal{V}(T_0)\}. \quad (6)$$

We normalize novelty over $\mathcal{P}(T_0)$ and consequentiality over $\mathcal{V}(T_0)$:

$$\widetilde{N}(u) = \frac{N(u) - N_{\min}}{N_{\max} - N_{\min}}, \qquad u \in \mathcal{P}(T_0), \quad (7)$$

$$\widetilde{C}(v) = \frac{C(v) - C_{\min}}{C_{\max} - C_{\min}}, \qquad v \in \mathcal{V}(T_0), \quad (8)$$

where $N_{\min}$ and $N_{\max}$ are the minimum and maximum novelty scores over $\mathcal{P}(T_0)$, while $C_{\min}$ and $C_{\max}$ are the minimum and maximum consequentiality scores over $\mathcal{V}(T_0)$. If either score has zero range, its normalized value is set to zero for all corresponding states.

The joint score for a candidate node v is defined as

$$J(v) = \lambda \widetilde{N}(\text{parent}(v)) + (1 - \lambda)\widetilde{C}(v), \quad (9)$$

where $\lambda \in [0, 1]$ controls the relative importance of novelty and consequentiality. We choose $\lambda > 0.5$ to assign relatively greater weight to novelty while retaining consequentiality as an important allocation criterion.

The novelty term evaluates the potential for generating a new continuation from the parent state, which is where the additional rollout will be initiated. The consequentiality term measures the magnitude of the observed change in outcome value associated with the decision leading to v. Thus, the joint score combines the potential for obtaining a novel continuation from the rollout's starting state with the importance of the corresponding reasoning decision.

4.3.4 Credit-Guided Allocation

We use the joint score to guide the allocation of the remaining k rollouts. For each candidate node $v \in \mathcal{V}(T_0)$, we compute $J(v)$ using Equation 9.

For each of the remaining k rollouts, we select the node with the maximum joint score:

$$v^* = \arg \max_{v \in \mathcal{V}(T_0)} J(v). \quad (10)$$

We then generate an additional trajectory from $\text{parent}(v^*)$, the state at which the decision leading to v^* was made. The novelty component therefore evaluates the potential for a new continuation from the actual rollout starting state, while the consequentiality component reflects the observed importance of the selected decision.

The original trajectory is preserved, and the new trajectory provides additional counterfactual evidence from the same decision state. After each newly generated trajectory is added, we update the relevant novelty, value, consequentiality, and joint-score estimates, recomputing the normalization statistics over the updated eligible sets. This ensures that subsequent allocation decisions incorporate the newly observed evidence. The process continues until all k reserved rollouts are consumed.

4.4 Final Credit Assignment and Policy Optimization

After counterfactual expansion, the tree values are recomputed to incorporate all n trajectories. The complete tree is then used to obtain variable, node-level credit estimates.

Rather than assigning a uniform trajectory-level advantage to every reasoning decision, we derive credit signals from the tree structure and local value differences. These signals are incorporated into the advantage estimates used for policy optimization.

Thus, the final training procedure is a policy optimization method with *tree-derived, variable-credit advantage estimates*. The credit assigned to an intermediate reasoning state can therefore depend on its observed position and consequences within the completed tree.

4.5 Algorithm

Algorithm 1 summarizes the complete rollout allocation procedure.

The procedure uses exactly $n - k$ exploratory trajectories and k targeted trajectories, maintaining the fixed total rollout budget n.

5 Dataset and Experimental Setup

5.1 Datasets

For RL training, we use 5,504 problems from the 30k training file released by Hou et al. (2025). A tree whose leaves are all correct or all wrong yields zero advantage under tree-based credit, so we filter by difficulty: the starting model answers each candidate problem 8 times, and we keep only those solved 1–7 times. Problems that also appear in a test set are removed. The inference-only study in Section 6.1 uses a random, unfiltered sample of 500 Omni-MATH problems (Gao et al., 2024).

We evaluate on four benchmarks totaling 1,755 problems: MATH500 (Hendrycks et al., 2021; Lightman et al., 2024), AMC (83 problems from

Algorithm 1 Novelty-Aware Credit-Guided Rollout Allocation

Require: Problem x, rollout budget n, reserved budget k, weight λ

Generate $n-k$ trajectories using local-entropy-guided expansion to construct T_0
for each eligible node $v \in T_0$ **do**
  Compute novelty $N(v)$ and consequentiality $C(v)$
end for
Normalize scores and compute $J(v) = \lambda\widetilde{N}(v) + (1-\lambda)\widetilde{C}(v)$
for $t = 1, \ldots, k$ **do**
  $v^* \leftarrow \arg\max_{v \in \mathcal{V}(T_0)} J(v)$
  Generate a trajectory from $\mathrm{parent}(v^*)$ and update T_0
  Update $N(v)$, $C(v)$, and $J(v)$ for all eligible nodes
end for
Recompute tree values and node-level credit signals
Perform variable-credit policy optimization
return Completed tree and updated policy

2022–2023), Omni-MATH-500 (Gao et al., 2024), and OlympiadBench (He et al., 2024). Evaluation relies on binary rewards where the final `\boxed{}` answer is matched against the reference using `math-verify`. Extended details regarding source mixtures, test-overlap filtering, and problem difficulty distributions are deferred to Appendix A.

5.2 Model and Training

Model. All experiments use Qwen2.5-Math-1.5B-Instruct (Yang et al., 2024) with its default chat template. We sample at temperature 1.0 and top-p 0.95. At the paper's temperature of 1.2, this model produced degenerate output in 13% of samples. Responses are capped at 3,072 tokens.

Methods compared. All methods share the same code path, training data, and hyperparameters, and differ only in how responses are sampled:

- **TreeRL** (Hou et al., 2025): EPTree with $(M, N, L, T) = (6, 2, 1, 2)$, i.e. 30 leaves per problem. All 30 are trained on with TreeRL's tree advantages (global plus local value differences, scaled by $1/\sqrt{|\mathcal{L}(v)|}$).
- **ChainRL**: 16 independent chains per problem through the same code. Without branching, the tree advantage reduces to a leave-one-out baseline.
- **Ours**: Phase I builds an EPTree with $(4, 2, 1, 2)$, giving $n-k = 20$ leaves. Phase II then allocates the remaining $k = 10$ rollouts with novelty horizon $h = 8$ and $\lambda = 0.5$. The total budget $n = 30$ matches TreeRL, and the completed tree is credited with the same tree advantages.

Run	Phase I tree	$n-k$	k	h	λ
ChainRL	16 chains	16	–	–	–
TreeRL	$(6, 2, 1, 2)$	30	–	–	–
Ours (main)	$(4, 2, 1, 2)$	20	10	8	0.5
Ablation 1	$(2, 2, 1, 2)$	10	20	8	0.5
Ablation 2	$(4, 1, 1, 3)$	16	14	20	0.5
Ablation 3	$(4, 2, 1, 2)$	20	10	8	0.3

Table 1: Rollout configurations. Every tree-based run generates and trains on $n = 30$ responses per problem. Ablation 1 shifts budget from Phase I to Phase II, ablation 2 changes the split and the novelty horizon h, and ablation 3 puts more weight on credit ($\lambda = 0.3$).

	TreeRL paper	Ours
Policy	7–14B	Qwen2.5-Math-1.5B-Instruct
RL steps	∼300	150
Learning rate	1.5e-6	1.5e-6, 10-step warmup
Problems / step	16	16
Trained resp./problem	30 Tree, 16 Chain	30 Tree and ours, 16 Chain
KL β	10^{-4}	10^{-4}
Temperature, top-p	1.2, 0.95	1.0, 0.95
Max response tokens	8,192	3,072
Optimiser	ZeRO-3, CPU Adam	ZeRO-2, GPU Adam

Table 2: RL settings: the TreeRL paper and ours. Each step uses 16 problems and one optimiser update.

We also run three ablations of our method (Table 1). Table 2 lists the shared settings next to the paper's.

Evaluation protocol. Our main metric is greedy accuracy, the paper's metric. We also report mean@8 and pass@8 over 8 samples per problem at temperature 1.0 (Appendix C). Checkpoints are scored every 50 steps. Models are compared on the same problems with a paired bootstrap: 2,000 resamples of problems, 95% CI.

6 Results

6.1 Phase I: surprisal-guided exploration

Phase I of our method builds the exploratory tree with EPTree's surprisal criterion. We replicate EPTree as an inference-only study on 500 Omni-MATH problems (Table 6, Appendix B), comparing EPTree, random forking, and i.i.d. chains. Our

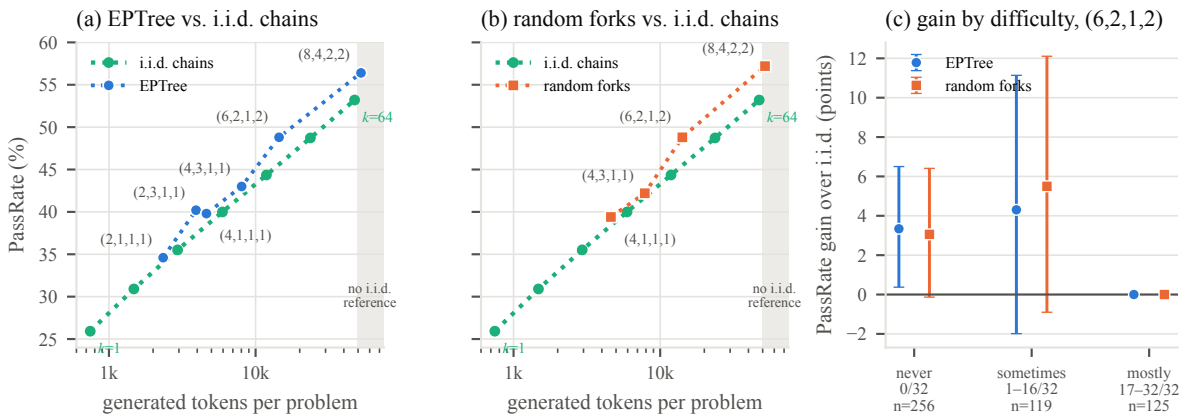


Figure 1: Tree sampling vs. i.i.d. chains on 500 Omni-MATH problems. (a,b) PassRate (at least one correct response) against generated tokens per problem, for EPTree (a) and random forking (b), each against the same i.i.d. curve ($k = 1, 2, \ldots, 64$ chains). Each point is one (M, N, L, T) run; dotted lines join runs in order of token cost. A point above the i.i.d. curve reaches that PassRate with fewer tokens. The shaded band lies beyond 64 chains, where no matched i.i.d. comparison exists. (c) Gain of the $(6, 2, 1, 2)$ trees over i.i.d. chains at the same token budget, by problem difficulty, with 95% bootstrap CIs. Difficulty is how many of 32 held-out chains (chains 33–64) solve the problem. The i.i.d. reference uses the other 32 chains, so the two are independent.

findings successfully replicate the core result of Hou et al. (2025): tree sampling provides a more efficient search than i.i.d. sampling. At the same token budget, tree runs gain up to +3.3 points in PassRate over independent chains (Figure 1). These gains concentrate on harder problems that i.i.d. sampling rarely solves, and trees consistently produce more learnable groups (mixed outcomes) per token budget.

We also replicate the finding that fork placement (surprisal-guided vs. random) has negligible impact on PassRate, with all paired differences being statistically indistinguishable from zero. Surprisal serves as a reasonable signal for diversity, but it is a weak predictor of consequentiality (whether a branch flips correctness). Instead, the probability of a fork flipping the outcome falls steadily with its relative position in the response. This limitation directly motivates Phase II of our method, which does not rely on surprisal alone: it scores candidate states by their observed consequentiality $C(v)$, the change in estimated value across the decision, together with their novelty, and spends the reserved rollouts where they yield new counterfactual evidence.

6.2 RL with credit-aware rollout allocation

Main result. Our method and TreeRL generate the same 30 responses per problem and train on the same 480 responses per step; the only difference is where in the tree the rollouts are spent.

Step	Chain	Tree	Main	Abl. 1	Abl. 2	Abl. 3
50	47.29	47.69	47.07	47.46	47.46	47.24
100	46.78	47.07	47.35	47.58	**47.81**	47.52
150	47.52	48.21	**48.55**	47.64	47.46	47.98
Best	47.52	48.21	**48.55**	47.64	47.81	47.52

Table 3: Greedy accuracy (%) on all 1,755 test problems. The starting model scores 47.52. Main and Abl.: our configurations (Table 1). Bold: best at each step. Best: best evaluated checkpoint.

Sending part of the budget to high-credit, high-novelty branch points gives the best model in the study: **48.55%** greedy accuracy at step 150, ahead of TreeRL (48.21%) and ChainRL (47.52%) at the same step (Table 3, Figure 2a). Our main run also improves the most of any run between steps 100 and 150 (+1.20 points, against +1.14 for TreeRL and +0.74 for ChainRL).

Gains on every benchmark. At step 150, our main run matches or beats TreeRL on all four benchmarks (Table 4, Figure 2b), with the largest gains on AMC (+2.4) and MATH500 (+0.6). It beats ChainRL on every benchmark, by +2.0 on MATH500, +2.4 on AMC, and +1.03 overall.

Robust across configurations. At step 100, every one of our four configurations outperforms both TreeRL and ChainRL (8 of 8 comparisons, +0.28 to +1.03 points; Table 5). Across all per-benchmark comparisons from step 100 on, our

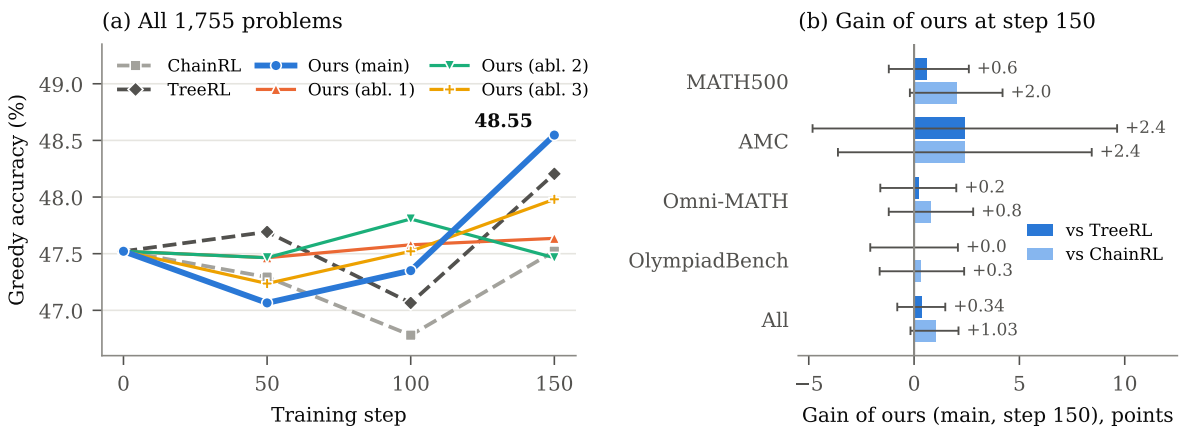


Figure 2: RL with Qwen2.5-Math-1.5B-Instruct, greedy accuracy on the 1,755 test problems. (a) Accuracy during training. Dashed lines are the baselines and solid lines our configurations (Table 1); step 0 is the starting model. (b) Gain of our main run over TreeRL and ChainRL at step 150, per benchmark and pooled. Whiskers are 95% paired bootstrap CIs.

	MATH500	AMC	Omni	Olymp.	All
Starting model	74.4	47.0	29.1	41.3	47.52
ChainRL @150	74.6	48.2	27.5	42.2	47.52
TreeRL @150	76.0	48.2	28.1	**42.5**	48.21
Main @150	**76.6**	**50.6**	28.3	**42.5**	**48.55**
Abl. 1 @150	75.6	48.2	**29.7**	40.1	47.64
Abl. 2 @150	76.2	49.4	27.5	40.7	47.46
Abl. 3 @150	**76.6**	45.8	27.3	41.2	47.98

Table 4: Greedy accuracy (%) per benchmark and on all 1,755 problems (All) for every configuration (Table 1), at step 150; ablation 3 at step 100. Bold: best in each column. AMC has 83 problems, so one problem is 1.2 points.

Comparison	vs TreeRL	vs ChainRL
Main @100	+0.28 [−0.80, +1.37]	+0.57 [−0.46, +1.60]
Abl. 1 @100	+0.51 [−0.63, +1.65]	+0.80 [−0.23, +1.88]
Abl. 2 @100	+0.74 [−0.40, +1.77]	+1.03 [−0.11, +2.11]
Abl. 3 @100	+0.46 [−0.68, +1.60]	+0.74 [−0.40, +1.82]
Main @150	+0.34 [−0.80, +1.48]	+1.03 [−0.17, +2.11]

Table 5: Paired differences in greedy accuracy (points) between our configurations and the baselines at the same step, with 95% bootstrap CIs.

method is ahead in 37 of 56. The gain comes from the allocation strategy itself, not from one tuned setting.

Largest gains on the hardest benchmark. Omni-MATH has the lowest starting accuracy (29.1%). On it, every one of our configurations is ahead of TreeRL at step 100, by +0.4 to +2.2 points (Figure 3). On hard problems, only a few intermediate steps decide whether a solution is correct. The consequentiality term $C(v)$ sends rollouts to exactly those steps, whereas TreeRL chooses its forks by token entropy alone and spends budget on uncertain tokens whether or not they matter for the outcome. As a result, TreeRL and ChainRL both lose ground on Omni-MATH during training (28.1% and 27.5% at step 150), and no baseline checkpoint after step 50 returns to the starting level. Our checkpoints do: the main run reaches 29.1% at step 100, and ablation 1 reaches 29.7% at step 150, the best Omni-MATH score of any checkpoint.

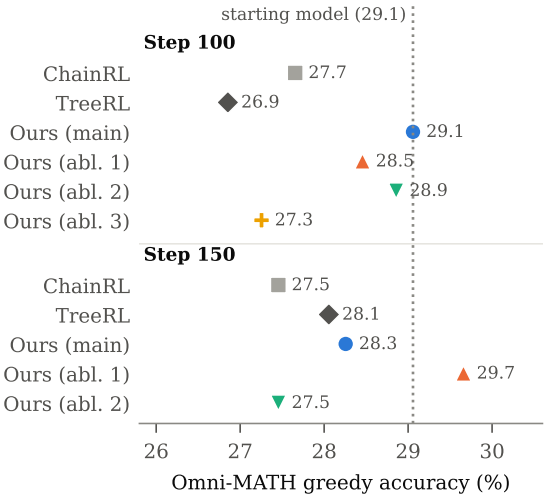


Figure 3: Omni-MATH greedy accuracy at each checkpoint. The dotted line is the starting model.

6.3 Ablations

Each ablation changes one part of the main configuration (Table 1). Every ablation outperforms both

baselines at step 100, and the main configuration gives the best model at step 150 (Table 3).

Phase I / Phase II split. Moving budget from the Phase I tree to Phase II (ablation 1, $10 + 20$ instead of $20 + 10$) lowers step-150 accuracy by 0.91 points [-2.11, $+0.34$]. A larger Phase I tree gives Phase II more leaves from which to estimate each node's value $V(v)$, so its consequentiality signal is better grounded. With the $20 + 10$ split, the main run gains the most of any run from step 100 to 150.

Novelty/consequentiality balance. Shifting the weight toward consequentiality (ablation 3, $\lambda = 0.3$) tracks the main run closely, $+0.17$ points at both steps 50 and 100, so the method is robust to the exact weighting. Its step-150 accuracy (47.98%, Table 3) stays 0.57 points below the main run, so the balanced $\lambda = 0.5$ remains the best choice. Both terms contribute: consequentiality finds the decisive steps, and novelty keeps Phase II from re-sampling continuations it has already observed.

Novelty horizon. With a longer horizon ($h = 20$) and a $16+14$ split (ablation 2), the model improves quickly and has the best step-100 score of any run (47.81%). The main configuration's short horizon ($h = 8$) with a $20 + 10$ split gives the best model at step 150, 1.08 points higher.

Every configuration we tried outperforms both baselines at step 100, so the method works across settings, and the main configuration ($20 + 10$, $h = 8$, $\lambda = 0.5$) is the best choice for accuracy at step 150.

7 Limitations and Future Work

A framework, not a single method. Our central contribution is the two-phase structure itself: a fixed rollout budget is split into an exploration phase that builds a tree without looking at rewards, and an exploitation phase that uses the outcomes of that tree to decide where the remaining rollouts go. Each phase is defined only by its goal and the score used to pursue it, so both are plug-and-play. The configuration studied here, TreeRL-style surprisal for Phase I and novelty plus consequentiality for Phase II, is one instantiation among many. Phase I could instead use random forking (which matched surprisal on PassRate in our Phase I study), diversity-driven branching, or position-aware forking; Phase II could target value uncertainty, answer disagreement among children, or any other notion of where counterfactual evidence is most useful. Systematically comparing such combinations within the same budget-matched setup is a natural next step.

Our scores are heuristics, not ground truth. The signals we use are proxies for the quantities we actually care about, and none of them is the true signal for its intended purpose. Surprisal reflects uncertainty over the next token, not the downstream usefulness of a branch for credit assignment. The novelty score $N_h(v)$ compares exact token sequences over a short horizon h, so a new rollout can count as novel while expressing equivalent reasoning. The consequentiality score $C(v) = |V(v) - V(\text{parent}(v))|$ is computed from a small number of leaves and is therefore noisy, particularly deep in the tree. Future work can keep the framework fixed and explore better estimates of the same two quantities, for example semantic or embedding-based novelty, or consequentiality based on value uncertainty, child-value disagreement, or the expected value of an additional rollout. More broadly, novelty and consequentiality need not be the quantities each phase optimizes: researchers can define their own objectives for exploration and exploitation and plug the corresponding scores into the same two-phase procedure.

More ablations and broader evaluation. Our ablations vary a few settings at a time on a single 1.5B model trained for 150 steps, and many of the resulting differences have confidence intervals that include zero. More ablations are needed to isolate the contribution of each component: novelty-only and consequentiality-only Phase II, random allocation of the k Phase II rollouts, finer sweeps over λ, h, and the $n - k$ / k split, repeated runs with different seeds, longer training, and larger models. Our evaluation also focuses on mathematical reasoning, where deterministic rewards enable controlled experimentation; extending the framework to tasks with richer process rewards or interactive environments is left for future work.

References

Fei Ding, Yongkang Zhang, Yeling Peng, Youwei Wang, Guoxiong Zhou, and Zijian Zeng. 2026. Reducing credit-assignment variance through counterfactual reasoning paths. *arXiv preprint arXiv:2605.16302*.

Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, et al. 2024. Omni-MATH: A universal olympiad level mathematic benchmark for large language models. *arXiv preprint arXiv:2410.07985*.

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. 2024. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.

Zhenyu Hou, Ziniu Hu, Yujiang Li, Rui Lu, Jie Tang, and Yuxiao Dong. 2025. TreeRL: LLM reinforcement learning with on-policy tree search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*.

Yizhi Li, Qingshui Gu, Zhoufutu Wen, Ziniu Li, Tianshun Xing, Shuyue Guo, Tianyu Zheng, Xin Zhou, Xingwei Qu, Wangchunshu Zhou, Zheng Zhang, Wei Shen, Qian Liu, Chenghua Lin, Jian Yang, Ge Zhang, and Wenhao Huang. 2025. TreePO: Bridging the gap of policy optimization and efficacy and inference efficiency with heuristic tree-based modeling. *arXiv preprint arXiv:2508.17445*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*.

Hieu Tran, Zonghai Yao, and Hong Yu. 2025. Exploiting tree structure for credit assignment in reinforcement learning with large language models. *arXiv preprint arXiv:2509.18314*.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. 2024. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*.

Yijun Zhang, Fan Xu, Jiaxin Ding, Yule Xie, Shiqing Gao, Xin Ding, Haoxiang Zhang, Luoyi Fu, and Xinbing Wang. 2026. Information gain-based rollout policy optimization: An adaptive tree-structured rollout approach for multi-turn LLM agents. *arXiv preprint arXiv:2607.06223*.

Heming Zou, Qi Wang, Yun Qu, Yuhang Jiang, Lizhou Cai, Yixiu Mao, Ru Peng, Xin Xu, Weijie Liu, Kai Yang, Saiyong Yang, and Xiangyang Ji. 2026. TRACE: A unified rollout budget allocation framework for efficient agentic reinforcement learning. *arXiv preprint arXiv:2606.11119*.

A Extended Dataset Details

Training problems. For RL we use `train_30k`, the 30,000-problem file that Hou et al. (2025) release with their training script. Each problem comes with a final answer and no reference solution. By source tag, it contains WebInstruct (26%), NuminaMath (24%) and its cn_k12 subset (17%), Omni-MATH (13%), and smaller sources.

A tree whose leaves are all correct or all wrong yields zero advantage under tree-based credit, so it contributes no gradient. We therefore filter by difficulty. We sample 12,000 problems and drop the 236 whose text (ignoring whitespace and case) matches a problem in any of our test sets; `train_30k` contains 454 of the 500 Omni-MATH-500 problems and 98 of the OlympiadBench ones. The starting model answers each of the remaining 11,764 problems 8 times (temperature 1.0, top-p 0.95, up to 3,072 tokens): 43% are never solved and 10% always.

We keep the 5,504 problems solved 1–7 times out of 8; the starting model's mean solve rate on them is about 0.45. Each RL step draws 16 of them, so a 150-step run sees 2,400 problems, each once.

Inference-only study (Phase I). The EPTree study in Section 6.1 needs no training data. It uses 500 Omni-MATH problems (Gao et al., 2024), a random sample (seed 0) without any filtering, which is the same construction as the paper's Omni-MATH-500. We chose hard, unfiltered problems because PassRate (at least one correct response) saturates on easier sets: an earlier run on 243 accuracy-filtered OlympiadBench problems reached 95% PassRate with 16 i.i.d. chains, leaving no room to compare samplers. On our Omni-MATH sample the base model solves 26% of problems with one response and 53% with 64.

Evaluation benchmarks. We evaluate on four sets, 1,755 problems in total:

- MATH500 (Hendrycks et al., 2021; Lightman et al., 2024): 500 problems.
- AMC: 83 problems from the AMC 12A/12B contests of 2022 and 2023.
- Omni-MATH-500: a random sample (seed 0) of 500 Omni-MATH problems (Gao et al., 2024). We score 499 of them; one prompt exceeds the 1,024-token limit.
- OlympiadBench (He et al., 2024): 673 problems.

None of these problems appears in our RL training set.

Reward and grading. The reward is binary: 1 if the last `\boxed{}` answer matches the reference under `math-verify`, else 0. A response cut off at the length limit counts as wrong. During RL, every token of a truncated response additionally receives a -1 penalty, as in the released TreeRL code.

Sampler	(M, N, L, T)	Leaves	Tokens	PassRate	Gain [95% CI]	Tokens saved [95% CI]	Distinct	Mixed (%)
i.i.d.	$k = 4$	4	2.9k	35.5	–	–	2.51	18.3
i.i.d.	$k = 8$	8	5.9k	40.0	–	–	3.92	25.8
i.i.d.	$k = 16$	16	11.9k	44.4	–	–	6.00	32.6
i.i.d.	$k = 32$	32	23.7k	48.7	–	–	9.30	38.9
i.i.d.	$k = 64$	64	47.3k	53.2	–	–	14.41	45.0
EPTree	(2,1,1,1)	4	2.3k	34.6	$+0.6$ [-1.3, $+2.6$]	9% [-21, $+34$]	2.32	16.6
EPTree	(2,3,1,1)	8	3.9k	40.2	$+2.8$ [$+0.6$, $+5.0$]	36% [$+9$, $+55$]	3.56	26.6
EPTree	(4,1,1,1)	8	4.6k	39.8	$+1.4$ [-0.6, $+3.5$]	20% [-11, $+43$]	3.65	24.6
Random	(4,1,1,1)	8	4.6k	39.4	$+1.0$ [-1.0, $+3.2$]	15% [-16, $+41$]	3.63	23.8
EPTree	(4,3,1,1)	16	8.0k	43.0	$+1.1$ [-0.9, $+3.2$]	16% [-16, $+42$]	5.53	31.6
Random	(4,3,1,1)	16	7.9k	42.2	$+0.4$ [-1.6, $+2.5$]	7% [-28, $+32$]	5.38	29.8
EPTree	(6,2,1,2)	30	14.5k	48.8	$+3.2$ [$+1.0$, $+5.3$]	40% [$+15$, $+57$]	8.41	39.4
Random	(6,2,1,2)	30	14.2k	48.8	$+3.3$ [$+1.2$, $+5.6$]	41% [$+18$, $+60$]	8.07	38.4
EPTree	(8,4,2,2)	136	52.4k	56.4	–	–	21.07	48.4
Random	(8,4,2,2)	136	51.9k	57.2	–	–	19.92	48.2

Table 6: All Phase I runs on 500 Omni-MATH problems. *Tokens*: generated tokens per problem, counted once per shared prefix. *Gain*: PassRate minus i.i.d. pass@k at the same token budget (interpolated log-linearly in tokens). *Tokens saved*: how many fewer tokens the tree uses than i.i.d. chains need to reach its PassRate. *Distinct*: distinct final answers per problem. *Mixed*: share of problems with at least one correct and one incorrect response. $(8, 4, 2, 2)$ exceeds the 64-chain budget, so it has no matched i.i.d. comparison. EPTree minus Random at the same shape (paired, PassRate points): $+0.4$ [-2.0, $+2.8$], $+0.8$ [-1.6, $+3.4$], 0.0 [-2.4, $+2.4$], -0.8 [-3.2, $+1.6$]. Mean per-response accuracy is 25.7–26.2% for every tree run, against 25.9% for a single i.i.d. chain.

Step	ChainRL	TreeRL	Ours (main)	Ablation 1	Ablation 2	Ablation 3
0	45.46 / 62.56	45.46 / 62.56	45.46 / 62.56	45.46 / 62.56	45.46 / 62.56	45.46 / 62.56
50	45.40 / 62.79	45.44 / 62.68	45.26 / 62.91	45.02 / 62.56	45.34 / 63.08	45.33 / 63.13
100	45.32 / 62.79	45.15 / 62.45	45.23 / 62.34	44.95 / 61.14	45.37 / 62.56	44.88 / 62.51
150	45.39 / 62.74	44.99 / 62.22	45.00 / 62.74	**45.51** / 62.85	45.39 / 62.68	–

Table 7: Sampled accuracy (%), mean@8 / pass@8, on all 1,755 test problems. Step 0 is the starting model. Ablation 3 is evaluated through step 100.

B Phase I: All Runs

C Sampled Accuracy

Table 7 reports mean@8 (accuracy averaged over 8 samples per problem at temperature 1.0, top-p 0.95) and pass@8 (share of problems with at least one correct sample) on all 1,755 test problems. Sampled accuracy is similar across methods. At step 150, ablation 1 has the highest mean@8 of any checkpoint (45.51), and at step 50 ablations 2 and 3 have the highest pass@8 (63.08 and 63.13).